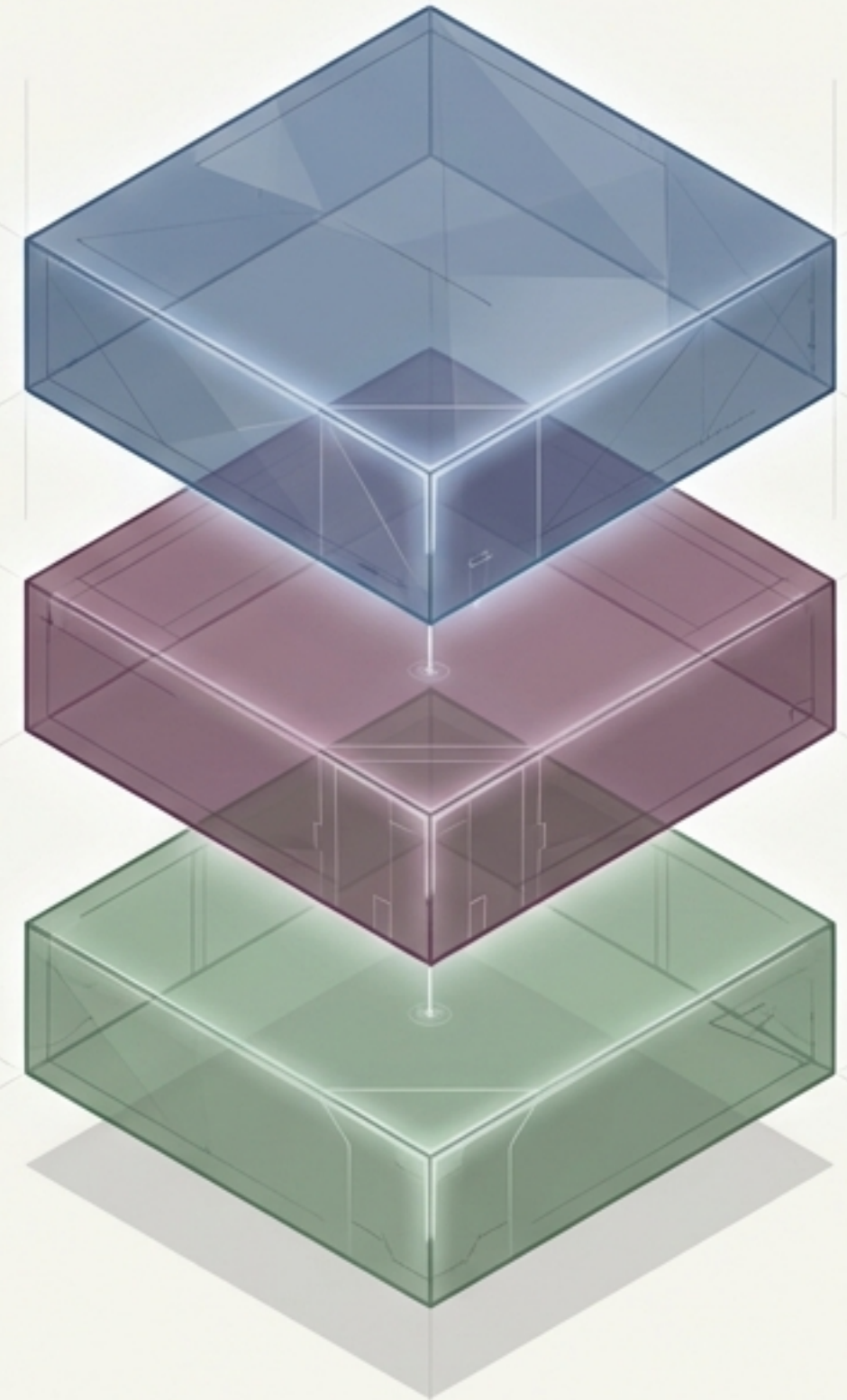


Refatoração Arquitetural: WebStore

Implementação e simplificação de código através dos padrões estruturais DAO e Repository.

Disciplina: Arquitetura de Software

Autores: Aaron Guerra Goldberg e Ramon Couto Santos



Categorias

	Categoria A - Descrição
	Categoria B - Descrição
	C

Produtos

	Produto 1 - Preço
	Produto 2 - Preço
	o 3 - Preço
	o 4 - Preço
	o 5 - Preço
	o 6 - Preço
	uto 7 - Preço
	Produto 2 - Preço
	Produto 2 - Preço

A Superfície

O sistema WebStore original funcionava perfeitamente para o usuário. Permitia o cadastro, listagem, edição e exclusão de Categorias e Produtos sem erros visíveis.

Os Bastidores

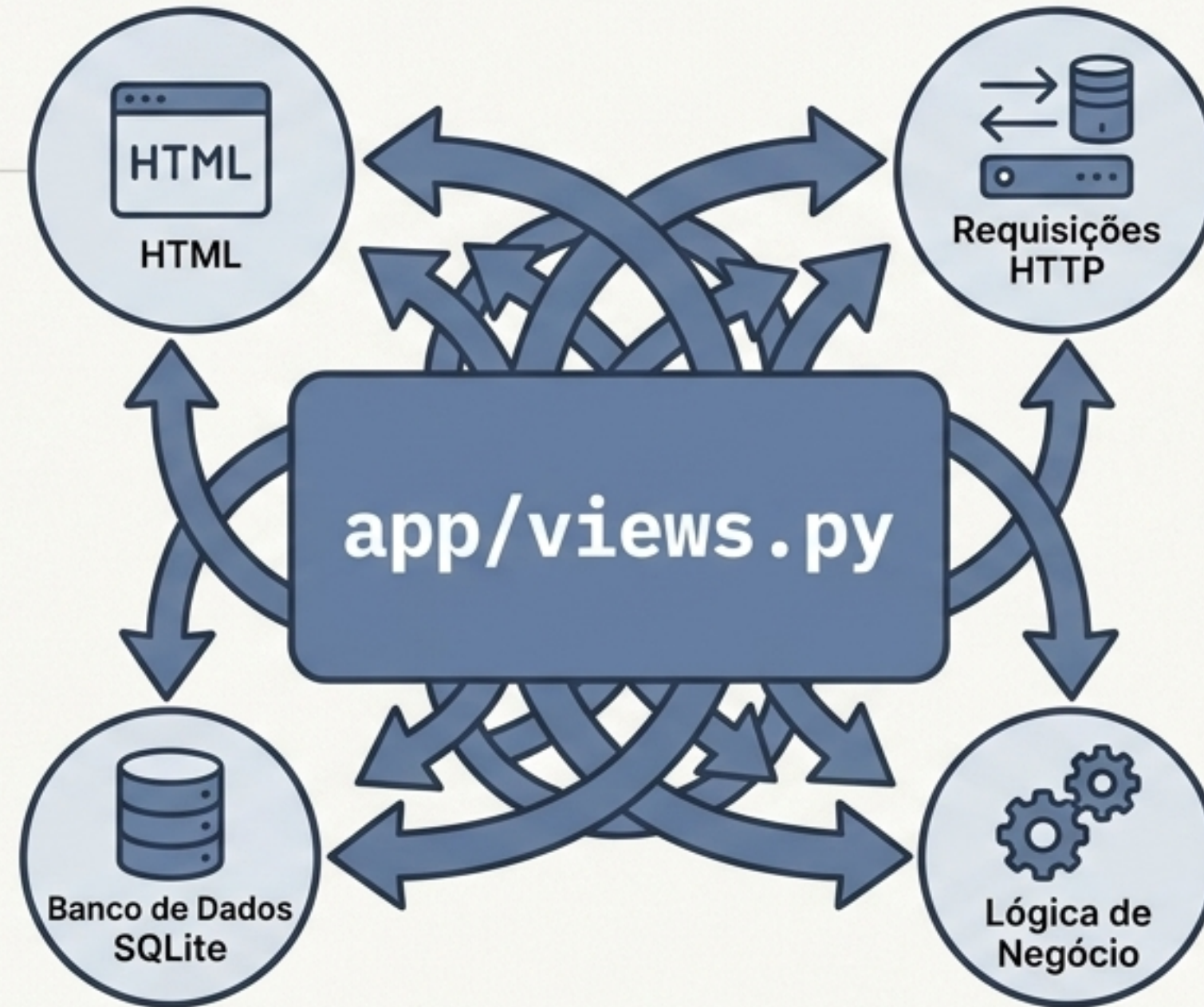
Tudo funcionava, mas a um custo estrutural altíssimo. O código por trás da aplicação estava centralizado de forma insustentável, criando uma base frágil e engessada.



O Deus views.py e o Acoplamento Extremo

SQL Misturado

Comandos SELECT, INSERT, UPDATE e DELETE inseridos diretamente no meio da lógica de tela.



Conexão Direta

A View abria e fechava conexões com o SQLite de forma manual a cada requisição.

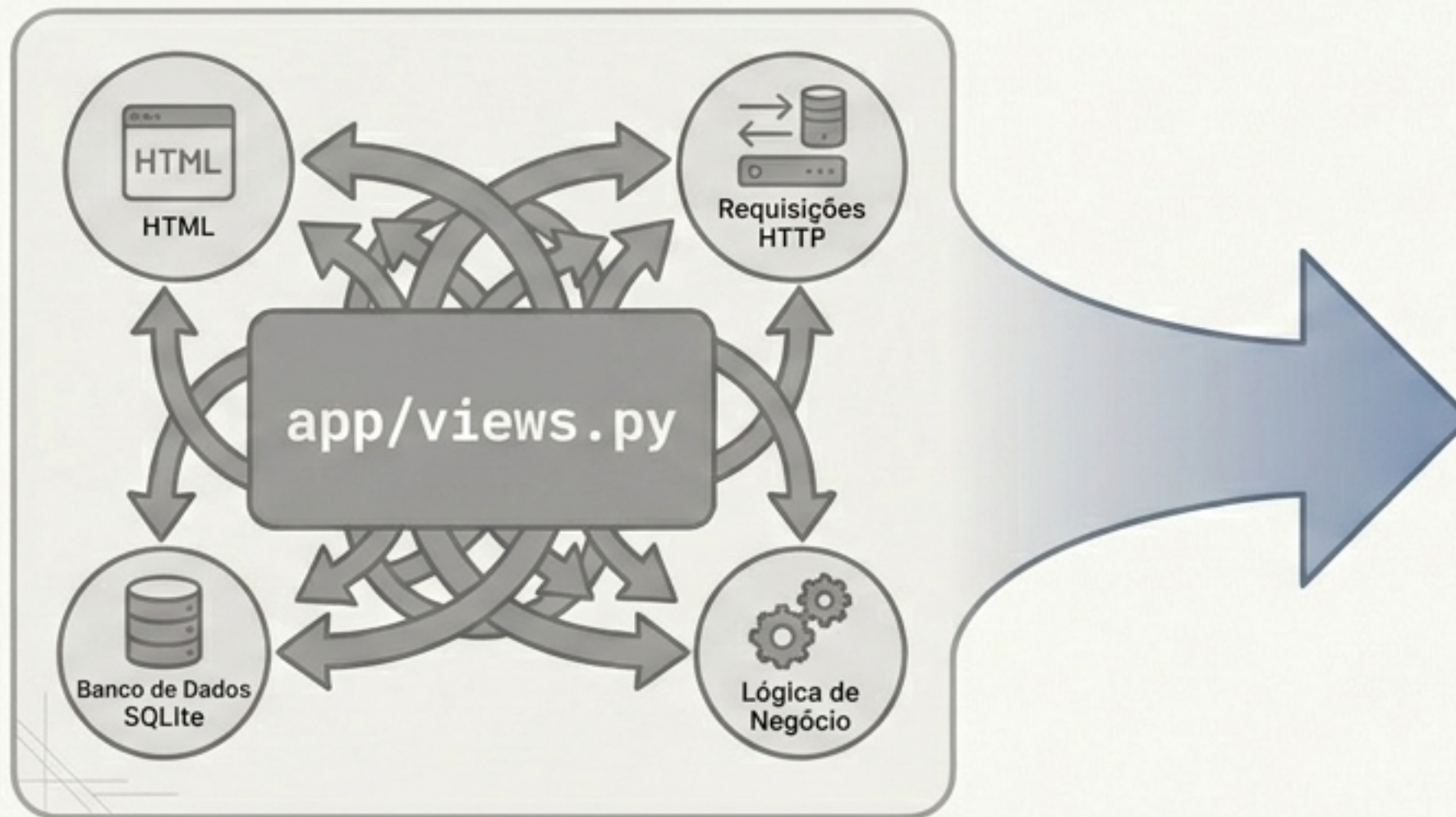
Sobrecarga de Fluxo

Recebia a requisição, decidia a ação, executava no banco, montava dicionários de formulário e renderizava o HTML. Tudo no mesmo arquivo.

O resultado prático? Código grande, repetitivo, frágil e difícil de manter. Qualquer mudança no banco ameaçava quebrar a interface.

Desacoplar para Conquistar

O objetivo central da refatoração é retirar todo o peso de processamento do `app/views.py` e distribuir as responsabilidades usando padrões de projeto pragmáticos.



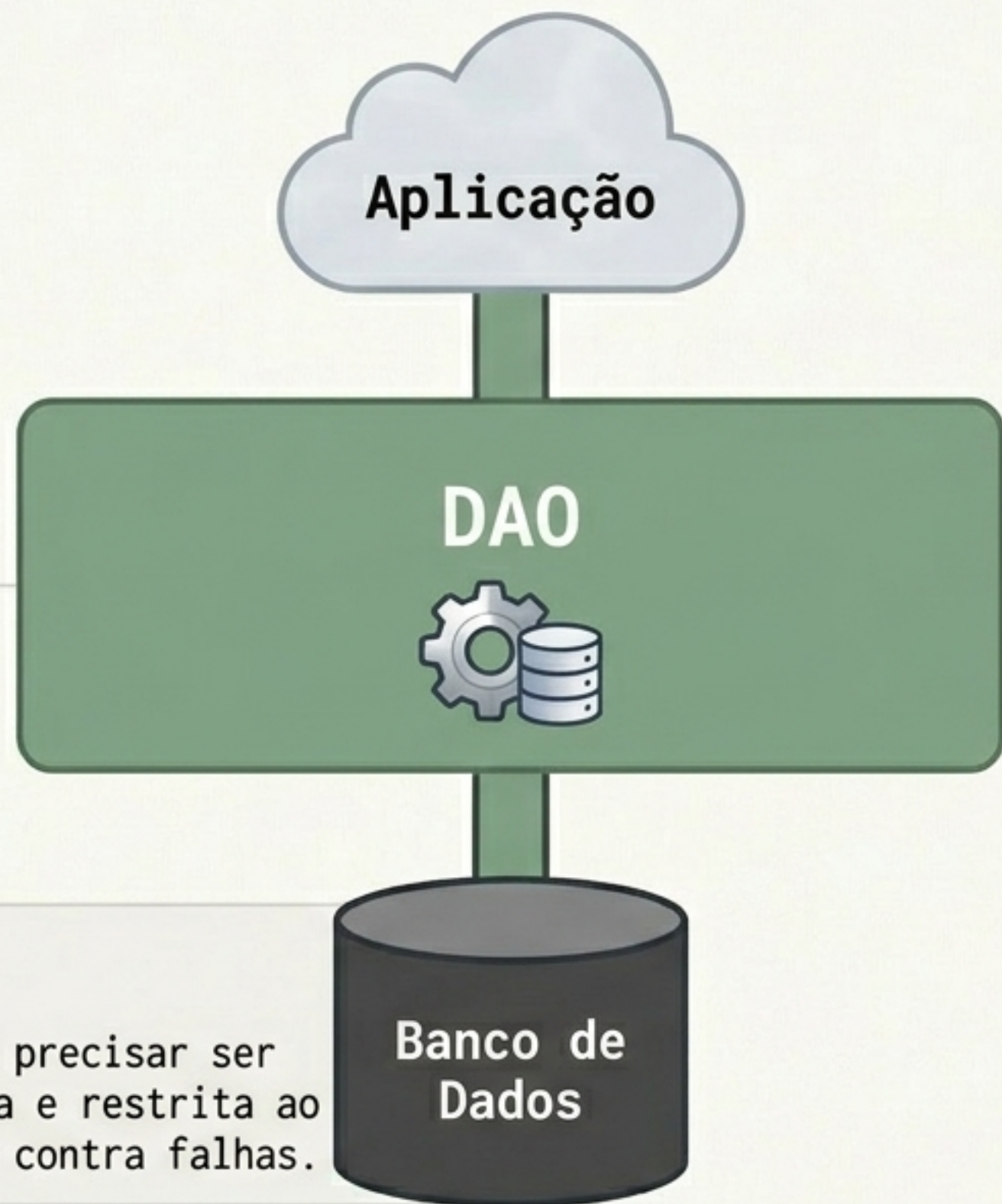
O Padrão DAO (Data Access Object)

O Que É

O objeto de acesso exclusivo aos dados. É a única parte de todo o sistema autorizada a 'conversar' com o banco SQLite.

O Valor Técnico

Se uma tabela mudar ou a consulta precisar ser otimizada, a alteração é cirúrgica e restrita ao blindando o restante da aplicação contra falhas.



A Função

Isolar e encapsular 100% das instruções SQL nativas do sistema.

Resoluções

Com o objeto de acesso a 100% e acesso a instruções SQL nativas do sistema, a solução é utilizada, a cada vez que o EPC de HTML e dados.

Implementação Física: app/dao.py

BancoDAO (A Base)

Centraliza o método `conectar()`. Conecta ao `db_solid.sqlite3`.

Destaque Técnico

Executa `PRAGMA foreign_keys = ON` para garantir a integridade referencial entre Produto e Categoria no SQLite.

CategoriaDAO (A Tabela)

Gerencia exclusivamente a tabela Categoria.

- Métodos: `listar()`, `buscar_por_id()`, `inserir()`, `alterar()`, `excluir()`.

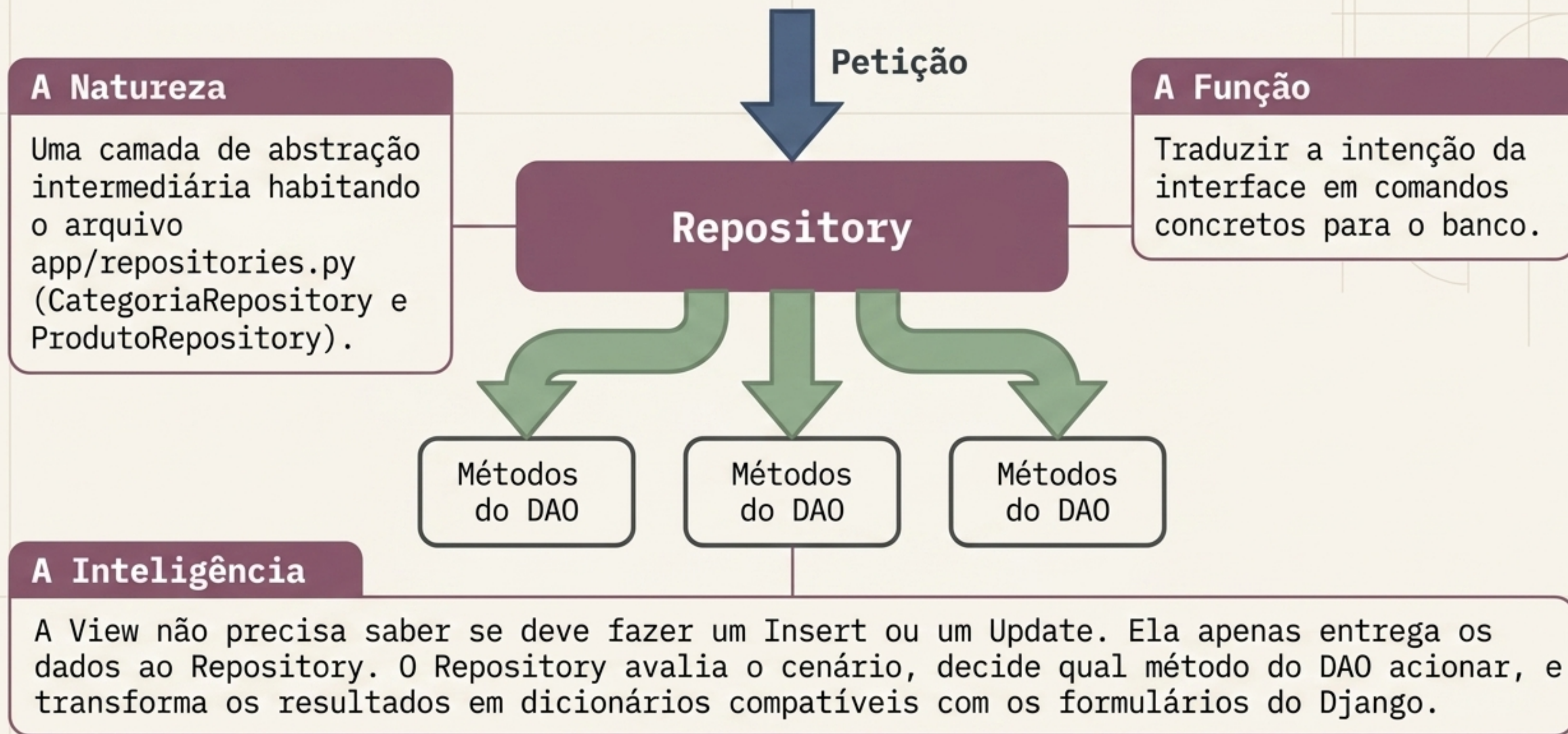
ProdutoDAO (A Junção)

Gerencia a tabela Produto.

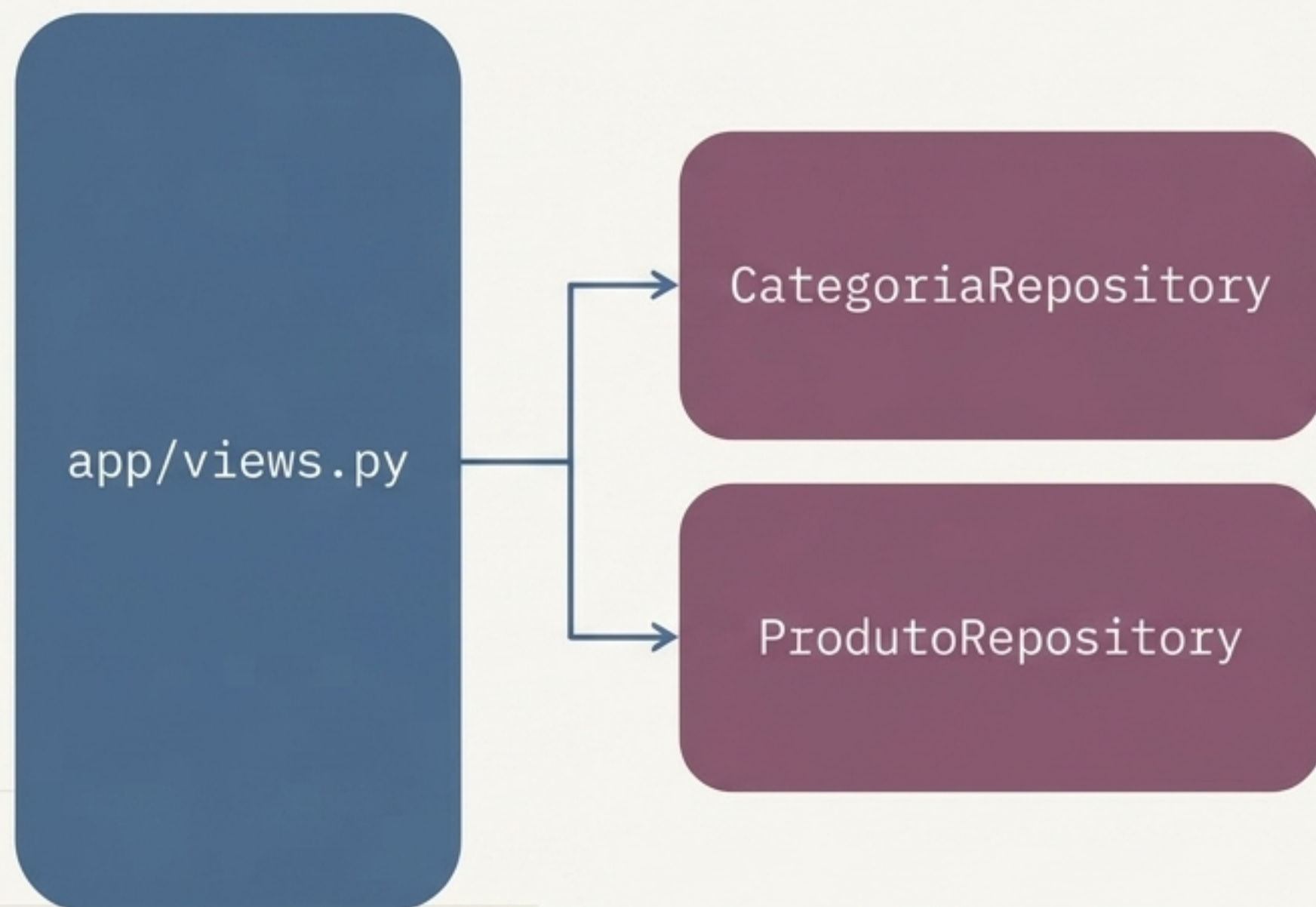
Destaque Técnico

Mantém o uso de `INNER JOIN` com a tabela de categorias para exibir o nome na tela, resolvendo a complexidade relacional sem repassar o problema para a View.

0 Padrão Repository (0 Intermediário)



O Novo Perfil do `views.py`



Step 1: Verifica qual rota/URL foi chamada pelo usuário.

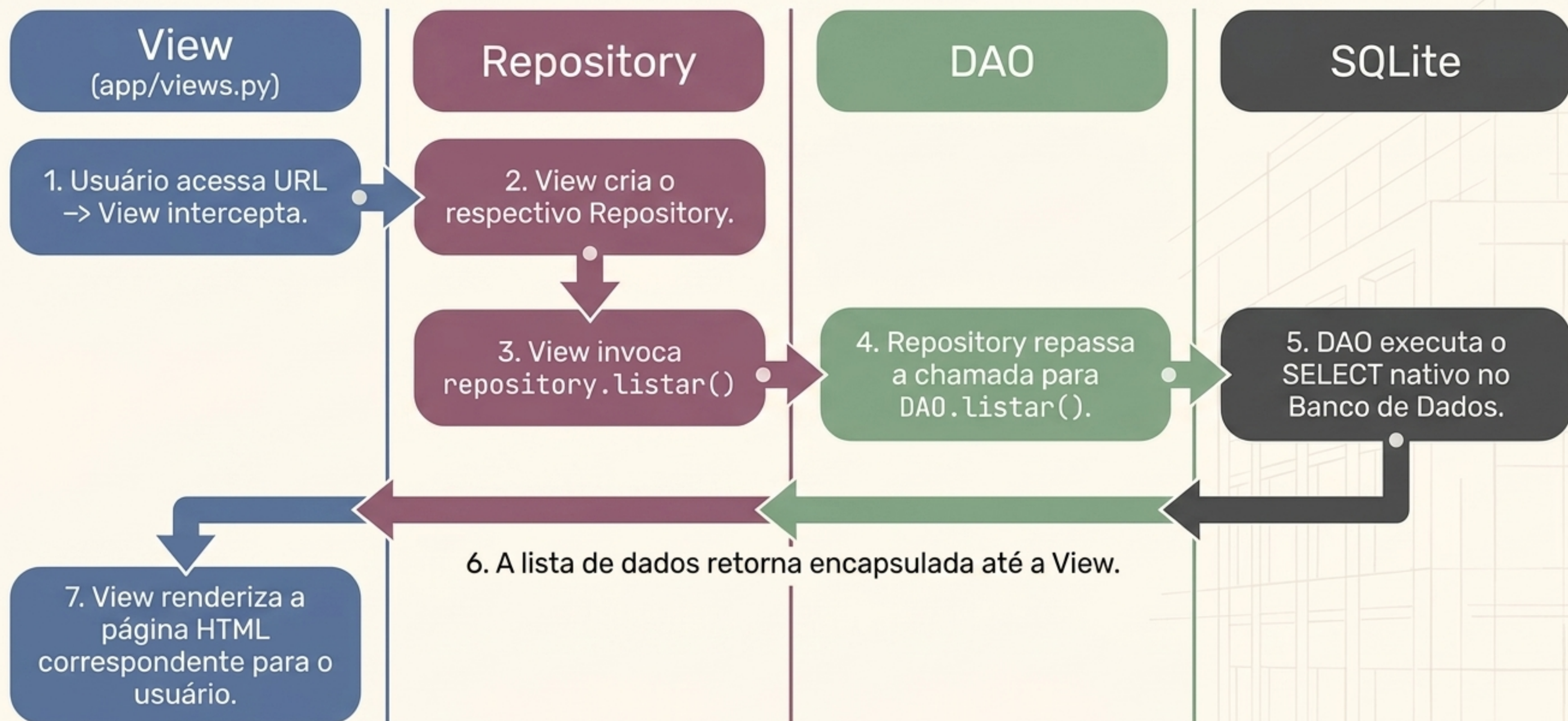
Step 2: Instancia o Repository correspondente (Categoria ou Produto).

Step 3: Repassa a requisição para o Repository processar.

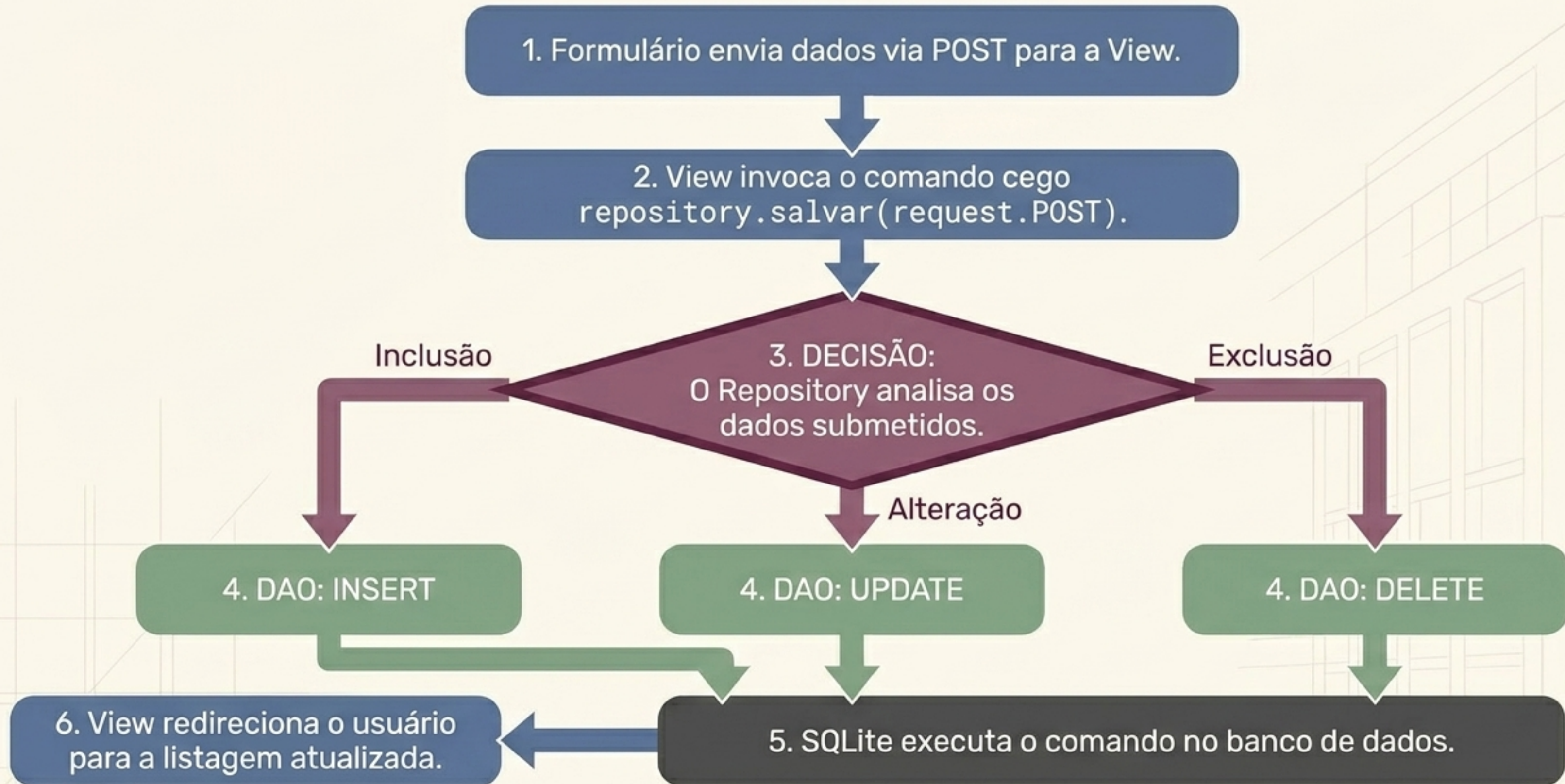
Step 4: Renderiza o template HTML correto ou emite o redirecionamento de tela.

“ A View voltou a ser apenas uma View. Sem SQL, sem conexões manuais, com foco 100% no fluxo da página.

Anatomia de Leitura: Fluxo de Listagem



Anatomia de Escrita: Lógica de Salvamento



Matriz de Refatoração: Antes vs. Depois

	O Sistema Original	A Nova Arquitetura
Localização do SQL	Misturado e espalhado na View.	Estritamente isolado nos arquivos DAO.
Gestão de Conexão DB	Aberta e fechada manualmente pela View.	Centralizada no método base do BancoDAO.
Escopo de Responsabilidade	Acúmulo de funções ('God Object').	Separação estrita (View / Repository / DAO).
Manutenibilidade	Difícil, engessada e propensa a erros.	Organizada, previsível e modular.
Impacto de Mudança	Alterar o banco exigia reescrever a View.	Alterar o banco afeta apenas o DAO.

Notas de Ambiente: Codespace e Validação

Embora a refatoração ocorra no core do Python, ambientes em nuvem exigem calibrações essenciais no arquivo `settings.py`.



ALLOWED_HOSTS

Ajuste obrigatório para aceitar o host dinâmico gerado pelo Codespace.



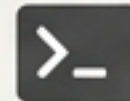
Configurações CSRF

Configuração de permissões de segurança para permitir transações e envio de dados via formulário.



Acesso por Portas

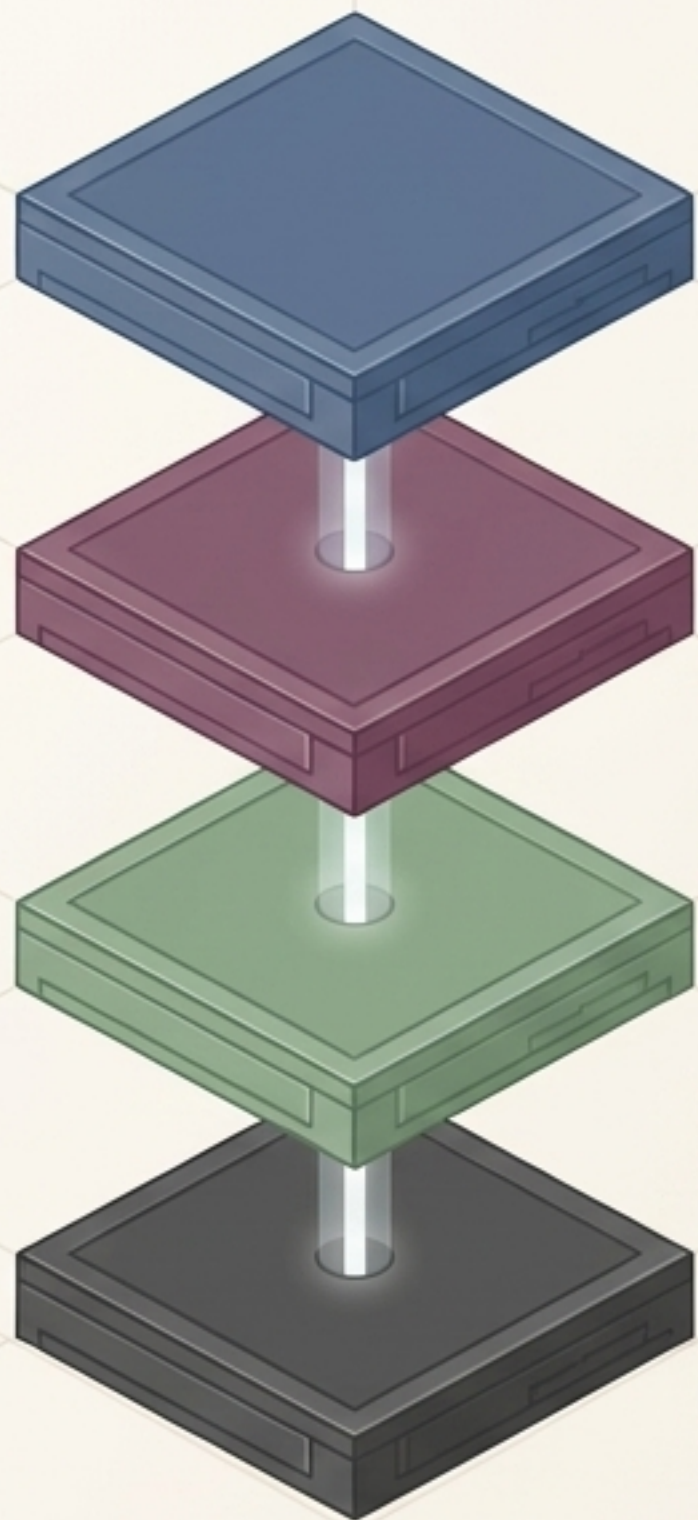
A visualização da aplicação em nuvem é roteada pelas portas encaminhadas (através da aba Ports do ambiente).



Comando de Diagnóstico Vital

Uso rotineiro de `python manage.py check` para validar imediatamente a integridade das configurações básicas do projeto Django.

Conclusão e Lições Arquiteturais



Padrões $\neq$ Complexidade

A aplicação de design patterns provou não ser sinônimo de código ilegível. Eles simplificam o fluxo desde que a separação de responsabilidades faça sentido pragmático.

Proteção contra Escala

Compreendemos que quando um projeto cresce, a arquitetura modular é a única barreira que impede o sistema de colapsar em um arquivo gigante e confuso.

Missão Cumprida

O sistema WebStore preservou 100% das suas funcionalidades originais para o usuário final, mas agora repousa sobre uma fundação técnica robusta, testável e pronta para o futuro.